

FocusFlow — Implementation Plan

Field	Value
Product	FocusFlow
Document	Phased Implementation Plan, v1.1 — includes Fast Track
Status	Draft for review
Date	1 October 2026
Source	<code>FocusFlow_PRD_v2.pdf</code> (Version 2.0)
Team	Solo or 1–2 people, building with AI assistance
Strategy	Fast Track first — 10-day prototype, then add features over ~20 weeks

***New here? Start with Section 1A below.** It is a 10-day prototype you can build immediately, written in plain language with no assumed experience. It is designed to become the foundation of the full app rather than being thrown away. Once you have read it, Section 1B explains why the remaining phases are arranged the way they are.*

1A. Fast Track — Your First 10 Days

What this is

A working version of the core idea: **the app builds your day and tells you why**. That is the whole product in miniature. You can build it in 10 days, show it to a few people, and keep building from there without starting over.

This section assumes you are learning to build with AI and have no prior experience with this kind of app. Everything is explained. Terms are defined when first used.

Words you will meet

Word	What it means here
Data	The information your app stores. A task, a time, a note.
Data model	The shape of that information. Which pieces belong together, and what each one is called.
Screen	One page or view the user looks at.
Function	A named piece of work your app can do. You can think of it as a machine with an input and an output.

Field	One piece of information inside a record. A task has a name, a length, a difficulty.
Timezone	The region your time belongs to. London, New York, Lagos. Matters more than you would expect.
Scheduler	The part of the app that decides what goes when. The heart of the product.
Block	A piece of time on the day. "09:30 to 10:15, write the report."
Fixed block	Time that cannot move. A meeting.
Reasoning	Why the scheduler put something where it put it. "This is hard work, you do your best thinking at 09:00."
Refactor	Rearranging code without changing what it does. Safe when code is tidy, risky when it is tangled.
Dead end	Code you will have to throw away and rebuild. The thing we are trying to avoid.
Retrofit	Add something to something that already exists. Usually easy if prepared for, expensive if not.

The three rules that make this reversible

Here is the important part, and it is short. Most of what you are skipping now is cheap to add later — **if** you do three things properly. Each costs about a day now and saves weeks later.

Rule 1 — Every task can belong to a goal, even if goals do not exist yet.

Right now you are not building goals. You are only building tasks. But when you create a task, give it an empty slot where a goal would go. Leave it empty. Ignore it.

That empty slot is the whole difference between adding goals in two days and adding them in three weeks. If the slot is not there from the start, you have to change every screen, every query, and every export to add it later.

The rule is simple: **leave room for the future, and do not build the future yet.**

Rule 2 — Meetings and blocks are the same thing, with a label.

Right now there are no calendar connections. Someone typing in "Standup, 09:30 to 10:00" is a fixed block.

Store it as a fixed block with a small note saying where it came from: "typed by hand". Later, when you connect a real calendar, meetings get stored the same way with a note saying "from calendar".

The scheduler should never care which. It only knows that time is taken. This single decision turns calendar sync from a rebuild into an addition, and it costs about an hour.

Rule 3 — Save the reasoning as pieces, not as a finished sentence.

This one is the most important and the easiest to get wrong.

The wrong way: the scheduler decides a task goes at 09:30, then writes the sentence "*Placed at 09:30 because your energy is high in the morning.*" You store the sentence.

The right way: the scheduler decides a task goes at 09:30, and stores the facts separately:

```
Placed at:      09:30
Your energy at 09:30: high
Task difficulty: high
Competing with: "Reply to contractor" (shorter, less important)
Blocked by:     "Fixed block, 10:00 to 11:00, typed by hand"
```

Then the app writes the sentence from those facts, whenever it needs to.

Why this matters: the weekly review and the insights features need the same explanations six months from now. If you stored finished sentences, the facts are gone and you cannot rebuild the explanation. You would have to re-derive it from data you no longer have.

Storing facts costs nothing today. Storing sentences costs weeks later.

One more, smaller: keep all your data access in one place. Every screen reads and writes data through the same few functions, rather than each screen doing its own thing.

This sounds like tidiness for its own sake. It is not. It means that later, when you want the app to work without internet, you change one place instead of rewriting everything. That is the difference between one day of work and one month.

The 10 days

Work roughly one task per day. Some days will run long; that is normal.

Day 1 — The shape of your data

Decide what a piece of information looks like. Write it down before you write any code.

You need five things:

A task. What the person wants to do.

- Name — what it is
- Length — roughly how many minutes it will take
- Difficulty — easy, medium, or hard
- Which goal it belongs to — **leave this empty** (Rule 1)
- Whether it's done

A block. A piece of time on the day.

- Start time
- End time
- What it is for
- Where it came from — hand-typed or calendar (Rule 2)
- Whether it can move or is fixed

Working hours. When the person is available to work. Start time, end time.

An energy curve. How alert the person is across the day. Three or four rough levels is enough: high, medium, low. They will draw it. It does not need to be accurate — it is a starting guess the app

improves later.

A day. One day's plan, made up of blocks.

Keep this written down somewhere permanent. You will refer to it constantly and it is what you build against.

Also today: build the setup screen. The person enters their working hours and draws their energy curve. You need this before anything else can work.

Days 2 to 4 — The scheduler

This is the heart. Three days.

Day 2 — List everything that takes time out of the day. Meetings, breaks, anything fixed. Then list the tasks that want to go in.

Day 3 — Sort. Walk through the available time and place work. Your rule: **hard work goes where energy is high, easy work goes where energy is low.**

You do not need the cleverest possible algorithm. You need one that is predictable, always gives the same answer for the same input, and can explain itself. Predictable matters more than optimal, because a user who moves something once and has it snap back tomorrow will stop trusting the whole thing.

Day 4 — Make it explain itself, and give it somewhere to say what did not fit.

Two things matter here:

- **Every placement has a reason**, using Rule 3
- **If everything does not fit, say so.** Name the specific tasks that did not get a slot. Never quietly drop something.

That second one is not a nice-to-have. If the app silently loses work, the person stops trusting it completely.

Days 5 and 6 — The day view

The screen people look at every morning. A timeline for the day, with blocks laid out on it.

- Show the blocks
- Let each block be tapped to show its reason (Rule 3)
- Let each block be marked done
- Show blocks that did not fit, separately and neutrally

Keep this screen simple. It is the one you will look at most, so it is worth doing properly rather than doing everything.

Day 7 — Letting people move things

When someone drags a block to a different time, the rest of the day has to be rearranged around it.

Expect this to take longer than you think. It is not snapping to a grid — it is running the whole placement again while treating the moved block as fixed. If you did Days 2 to 4 properly and kept the

placement logic in one function, this is easy. If the logic is spread through your screens, this is painful. This is why the plan insists the scheduler lives in one place, separate from the screens that use it.

Day 9 — The evening review, and making it not fall over

The unglamorous day that prevents bad days later, plus the one feature added after review.

- **The minimum journal.** `JournalEntry` plus an evening review that reads the day's real `DayPlan`, `PlanItem`, and `Task` rows and shows what actually happened. One free-form text box, a five-tap mood row, and a read-only list of past entries. No prompts, no coach, no search — those need the AI coach, which is not in this prototype. See Section 2.2 of the technical design for why this table is five fields wide.
- **Timezones and daylight saving.** Do not skip this. Almost every scheduler bug you will ever chase comes back to time handling. Store the actual timezone, convert deliberately, never store a bare time like "09:30" without saying what timezone it is in.
- What happens if the app closes mid-save
- The words on every screen, checked against the PRD's tone rules

Why the journal lands here and not in Phase 7. Day 10 is the tester session, so anything not finished by the end of Day 9 is never tested by a real person. PRD risk 8 says outright that *"journaling is abandoned in week two."* The question a prototype can answer is not whether journaling is well built — it is whether anyone writes in it at all.

Day 8 — Replanning, and the empty screens

When the day is not finished, the app proposes a new one.

This matters more than it looks. It is the moment a person decides whether to come back tomorrow or give up.

Three rules:

- No guilt. No red. No "you failed." No count of missed days.
- Say what happened plainly: three tasks moved to tomorrow, here is why.
- Accept with one tap.

Read the PRD's tone section before writing these words. This screen carries the emotional weight of the entire product.

Also on this day: **empty states** — what each screen shows when there is nothing to show. Moved here from Day 9 to make room for the journal. A first-run user who meets four blank panels concludes the app is broken, and this is the cheapest possible fix.

Day 10 — Get it in front of someone

Prepare three realistic test days with different shapes:

- 1.
- 2.
- 3.

Write a short script of what to ask. The most important question is on Day 4 of the plan: **"When you looked at this schedule, did you notice anything different about when the work sits? Tell me about it."**

If they cannot tell, the core idea is not landing. That is worth knowing now, after 10 days, rather than after six months.

What you have after 10 days

Working	Not working yet
The app builds a day from your tasks	Goals — tasks have an empty slot waiting
Work is placed by energy, not just order	Quick capture — tasks are typed in
Every placement explains itself	Focus mode — no protection from distractions
You can move things and it rearranges	Evening review, journal, routines
Replans without making you feel bad	Calendar sync — meetings typed by hand
	Progress, insights, offline

What you can and cannot learn from this

You can learn: Does arranging work by energy feel different from putting it in order? Do people believe a day they did not build themselves? Does showing the reason make the plan trustworthy? Does replanning without guilt bring people back? Do people move fewer than 30% of blocks?

You cannot learn: anything about habits, keeping people, goals, or reflection. You have half the product. This tests the idea, not the product.

Before the test, one instruction

Ask each person to type in tomorrow's meetings the night before. Without that, the app will schedule over their real calendar and they will blame the app for a problem you already knew about. This will produce a false result.

On the 30% number

Fewer than 30% of blocks moved is the signal that the scheduler is working. More than that and people do not believe it. If they are moving half the blocks, do not add features — fix the placement.

This number matters more than any feature you could add next.

1B. What Comes After the Prototype

The rest of this plan covers the full build. If you start with the Fast Track above, use this section as your roadmap for what follows.

How hard is each dropped feature to add later

Feature	Difficulty	Time	Why
Goals and Projects	Easy	1–2 days	Only easy if Rule 1 was followed. Otherwise 2–3 weeks
Quick capture and inbox	Medium	4–6 days	The capture parsing is the work, not the storage
Focus mode	Medium	4–6 days	Mostly independent. App blocking is limited by the phone
Evening review and journal	Medium	6–8 days	New screens, but everything they reference exists
Routines	Medium	4–5 days	Needs a repeat rule on tasks
Onboarding	Easy	3–4 days	Just a flow writing into structures you have
Calendar sync	Medium-hard	8–12 days	Real work, but cheap architecturally if Rule 2 was followed
Progress and insights	Medium	8–10 days	Needs weeks of data to be honest
Offline support	Hard	3–4 weeks	The one genuinely expensive addition

About offline. Adding offline support to an app that was built online-only is close to rewriting the data layer. You can reduce the cost to about a day by keeping all data access in one place, then later adding a local copy behind it. Whether you need offline at all is a real product decision — many useful apps launch online-only and add it after they see demand.

The order to add them

Build the prototype first (days 1 to 10). Then:

Weeks	Add	Why here
3–4	Goals and Projects	The spine. Everything makes more sense once tasks point somewhere
5–6	Quick capture and inbox	Removes the biggest friction cost. This is what users feel first
7	Onboarding	Cheap, and you want it before strangers rather than friends
8–9	Evening review	Completes the loop. Without it you have planning but no habit
10–11	Calendar sync	Real days are flowing now, so real calendars matter
12–13	Journal and routines	Build on the daily loop
14–16	Focus mode	Execution support. Harder than it looks because of phone limits
17–19	Progress and insights	Needs accumulated data to be honest
20+	Offline, if still wanted	Decide deliberately, not by default

Roughly 20 weeks to a complete product on top of the prototype. Slower than the full plan's 22 to 27 weeks, because this route pays two small taxes — you are rebuilding goals and capture that the full plan had in week 4.

Dead ends — four things that would cost a rebuild

Scheduling logic written inside the screens. It tangles into components within days. Keep it in one function that takes your tasks and constraints and returns the day. When you add calendar sync later, you change what goes in, not the thing that works it out.

Reasoning saved as finished sentences. Covered in Rule 3. The one most people get wrong, and the only one that cannot be undone by effort.

Time stored without a timezone. Every scheduler bug traces back to this. Store real timezones and convert deliberately.

A meeting modelled as "a task that cannot move." This feels efficient and it is a trap. Meetings and time blocks are different things, and treating them as the same makes calendar sync a rebuild.

Building with AI — the honest version

What AI is genuinely good at: the repetitive work. Forms, screens, lists, saving data, the boring 60% of any feature. This is real leverage, and it is part of why 10 days is realistic.

What AI is bad at: deciding whether the product works. It will not tell you the day planner produces plans nobody trusts, that the energy curve is backwards, or that you have quietly dropped something you needed. It builds what you ask for and stays quiet about what you did not.

Two habits that follow from that:

Keep a decision log. A plain text file. Every choice worth remembering, one line, with the date. Three months from now you will have forgotten why reasoning is saved as pieces, and you will be tempted to tidy it into sentences. A written note is what stops that.

Only keep code you can explain. If you cannot explain a line, you cannot fix it when it breaks at 11pm. A 10-day build is small enough that you can genuinely understand all of it. That is an advantage — do not spend it.

1. How to Use This Plan

This plan converts the PRD into an ordered sequence of phases. Each phase answers four questions:

- **What are we building?** — the scope, and which requirement IDs it delivers
- **Why now?** — why this phase sits before the next one
- **How do we know it works?** — exit criteria that are verifiable, not subjective
- **What do we check?** — the check-in ritual before moving on

1.1 The Check-In Ritual

You asked for a check-in after every phase so problems surface before they compound. This is the single most valuable habit in the plan, because the failure mode of this product is subtle: things work, but do not feel right, and the defect compounds silently through later phases.

Every phase ends with a check-in that takes 30 to 60 minutes. It has four parts, performed in order:

#	Step	Duration	What you do
1	Run the exit criteria	10 min	Verify every exit criterion objectively. A criterion you cannot verify does not pass.
2	Live the day	15 min	Use the product as a real user for one full day. Plan in the morning, work, review in the evening. Not a test — a day.
3	Note friction	10 min	Write down every point where you hesitated, wanted to go back, or felt stupid. Raw observations only, no fixing yet.
4	Decide	15 min	Fix, defer, or reverse. Reversing is a valid outcome and should be cheap to choose.

Three rules make this work:

- **Fix before proceeding.** Do not carry a known defect into the next phase. It will be built around and become expensive to remove.
- **Reversal is cheap.** Every phase should produce something standalone enough to abandon without wasting later work. If a phase cannot be abandoned cheaply, it is scoped wrong.
- **Notes are evidence.** Write observations down during step 3 before discussing them. Discussion before writing produces rationalisations.

1.2 What Each Phase Produces

Every phase delivers four artefacts, not just working software:

Artefact	Purpose
Working software	The feature itself
Requirement coverage note	Which requirement IDs are now satisfied, and which were cut
Friction log	Accumulated observations from check-ins, carried forward
Decision record	Any choice made that a future reader would question

1.3 Phase Overview

Phase	Name	Delivers	Est.	Gate
0	Resolve Blockers	Answers to the questions that prevent any building	3–5 days	All blockers answered

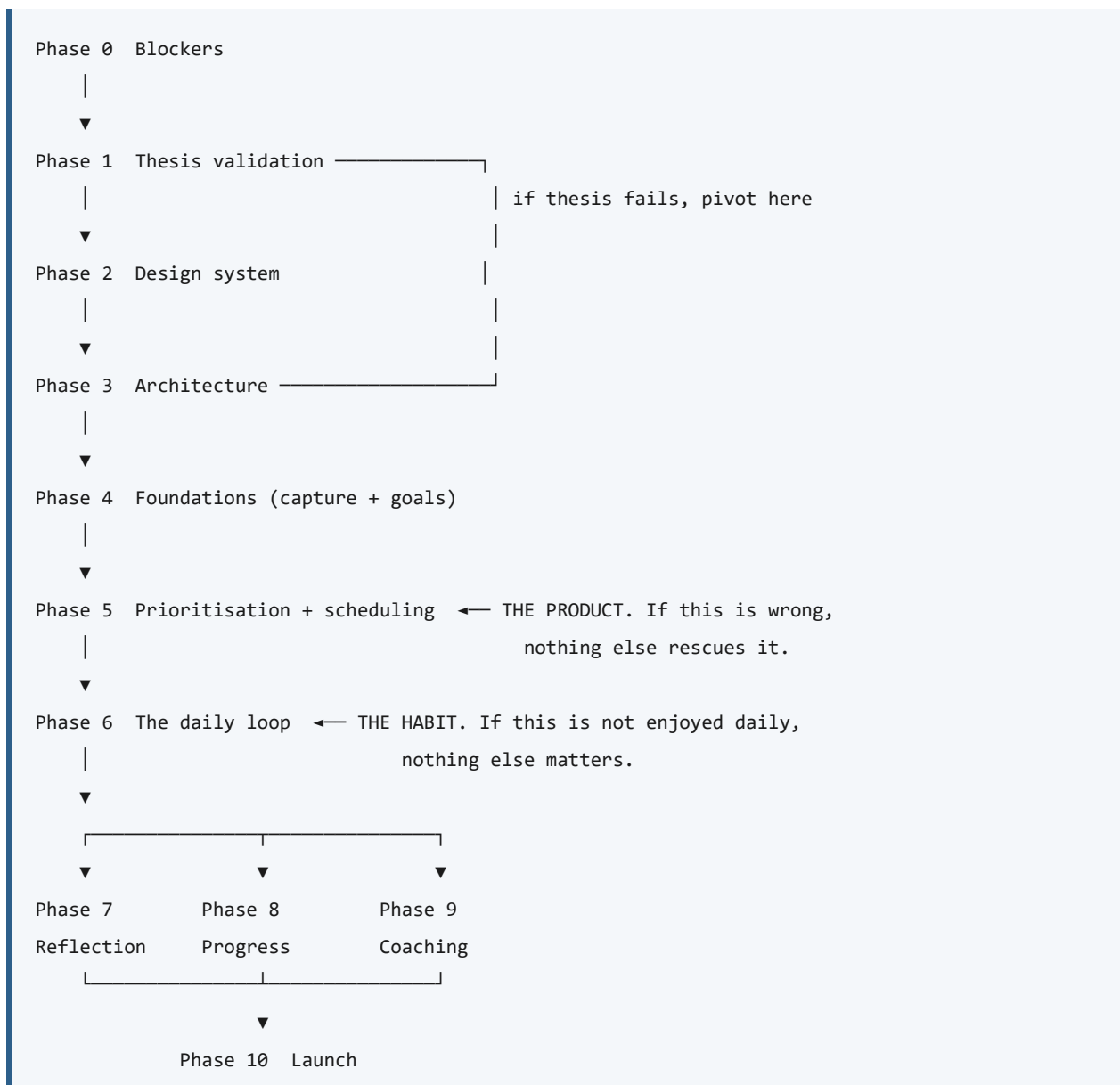
1	Product Thesis Validation	Proof that energy-aware scheduling is worth building	5–7 days	Thesis confirmed or pivoted
2	Design System	Tokens, components, patterns, accessibility baseline	10–15 days	WCAG AA foundation verified
3	Architecture	Platform, sync, data, AI boundaries, privacy decisions	5–8 days	All ADRs accepted
4	Foundations	Data model, capture, goals, projects, tasks	15–20 days	Core objects usable
5	Prioritisation & Scheduling	Ranking, energy-aware day plan, reasoning	15–20 days	70% block acceptance in testing
6	The Daily Loop	Morning plan, focus mode, evening review, replan	15–20 days	Full day works end to end
7	Routines & Reflection	Routines, journal, week continuity	10–12 days	Journal tied to real work days
8	Progress & Insights	Progress view, insight feed, weekly review, score	12–15 days	Course correction works
9	Coaching Layer	Morning briefing, overload, journal interrogation	10–12 days	Coach silent when nothing to say
10	Onboarding & Launch Readiness	Guided setup, notification policy, export, hardening	12–15 days	All 16 release gates verifiable
11	Beta & Validation	Real users, measurement, iteration	4 weeks	Retention thresholds approaching
12	Post-Launch	Leaderboard decision, V2 candidates	ongoing	Defer by default

Total estimate: roughly 22 to 27 weeks for a solo developer, excluding Phase 11 beta duration.

Two honest notes on these estimates. A solo developer with a day job typically produces 15 to 20 productive hours per week, which is why Phase 11 shows as calendar weeks rather than effort weeks. And Phase 1 is deliberately small — it exists to avoid the most expensive mistake in this plan, which is building a scheduling engine that nobody values.

2. Critical Path

Not all work is equally important. This is the sequence that determines whether the product works at all.



Phases 4 and 5 are load-bearing. Every later phase depends on the scheduling output being believable. If you run short on time, cut Phase 9 (coaching) to a thin version before touching Phase 5. A product without an intelligent coach is a lesser product. A product with an unintelligible schedule is nothing.

3. Phase 0 — Resolve Blockers

Duration: 3 to 5 days

Requirements delivered: None. This phase removes the reasons building cannot start.

Depends on: Nothing

3.1 Why This Is First

Open Questions 2 and 3 from the PRD are not merely unanswered — they are structurally blocking. The consistency score cannot be computed without deciding what it measures. The scheduling engine cannot be designed without knowing what signals place work in time. Building first and deciding later means rebuilding the scheduler, which is the most expensive code in the product.

3.2 Blocker 1 — Consistency Score Weighting (PRD Q2)

The score drives the weekly review, the strengths and weaknesses breakdown, and the leaderboard. A score that ranks users on a contested metric is unfair, and the leaderboard makes unfairness visible.

Option	Definition	Advantage	Problem
A. Task completion ratio	Completed ÷ scheduled in the week	Simple, immediately understood	Measures busyness, not progress. Rewards users who schedule trivial tasks
B. Goal progression	Tasks completed that advance an active goal	Measures what matters	Cold start problem — no history early on
C. Ritual adherence	Morning plan and evening review completion	Directly predicts retention	Rewards showing up rather than accomplishing
D. Weighted blend	Combination of the three	Balanced	Weights are arbitrary and hard to explain to users

Recommendation: option B, with a transparent formula, and option C tracked separately as a distinct private metric.

Reasoning: the product's entire thesis is that effort should serve goals. A score measuring task completion measures busyness, which is the exact failure mode the product exists to correct. A user scoring well on task completion while none of their goals advance has been underserved, and a score that says otherwise teaches the wrong lesson.

The blend problem is real, so the formula must be explainable in one sentence shown to the user:

Your consistency is the share of your scheduled work you completed that moved one of your active goals forward.

This satisfies requirement I3, that insights cite the behaviour they are based on. Ritual adherence should be measured privately for retention analysis, but must not appear in a user-facing score, because it invites gaming and does not reflect achievement.

Action required from you: confirm or override this recommendation. Then write the exact formula down, because requirement I6 requires the score to be reproducible from underlying task data.

3.3 Blocker 2 — Energy Signal Sourcing (PRD Q3)

The energy-aware scheduler is the core differentiator (Risk 11). This decision determines its architecture.

Approach	Signal source	Advantage	Problem
A. User-declared only	Energy curve entered in onboarding, manually adjustable	Zero cold start, immediately available, fully explainable	Static. Energy shifts week to week (Risk 6)
B. Observed from behaviour	Inferred from completion rates by time of day	Accurate over time	No data for new users. Requires 3 to 4 weeks before it works
C. Hybrid	Declared pattern, refined by observed data	Best of both; declared unblocks cold start	Most complex to build and to explain

Recommendation: option C, and this is the one place the added complexity is justified.

Cold start is the killer for pure observation. A new user has no history, and the first generated day is the moment belief forms (Success Criterion 2). If that first day is bad because we have no data, the user never reaches the point where data would help.

Declared data gives the first week something to work with. Observed data makes the fourth week noticeably better than the first. The user can feel this improving, which is itself motivating — the product visibly learns them.

Concretely:

- Onboarding asks for a simple visual energy curve, defaulting to a common adult pattern the user adjusts with a drag
- The declared curve is labelled in the UI as an assumption, not a fact
- Observed completion data by hour and task difficulty progressively adjusts it
- Every adjustment is disclosed: "We moved your deep work to mornings based on what you have actually been finishing" (satisfies Q8)
- Users can always override and can always revert to their declared pattern

Action required from you: confirm or override. Then define what happens when a user's observed data contradicts their declared pattern. Recommendation: observed data wins after a defined threshold, with disclosure and a permanent revert option.

3.4 Blocker 3 — Leaderboard Timing (PRD Q1)

Recommendation: **defer to post-launch**. Three reasons, in order of weight:

- 1.
- 2.
- 3.

This is easily reversed later. Include it in the design system's reserved patterns so adding it is not a redesign.

3.5 Exit Criteria

- Consistency score formula written down in plain language and approved

- Energy signal approach confirmed, including the contradiction rule
 - Goal time horizon decided (PRD Q4): recommendation is **quarterly**, because annual goals do not produce visible progress within a retention window, and quarterly progress reviews map onto the weekly review cadence
 - IR versus AI communication approach decided (PRD Q5): recommendation is to **lead with what the product does, not that AI does it** — "your day is arranged around when you think best" rather than "AI scheduled your day"
 - Free tier decision made by business (PRD Q8)
-

4. Phase 1 — Product Thesis Validation

Duration: 5 to 7 days

Requirements delivered: None formally. Validates the assumptions underpinning S2, I1, and Success Criterion 3.

Depends on: Phase 0

4.1 Why This Phase Exists

The product's central claim is that scheduling by energy rather than by clock makes a perceptible difference. If users cannot feel that difference, the entire differentiator is decorative and the product collapses into a slightly nicer to-do list competing against far more mature tools.

Risk 11 names this as a high-severity risk with a specific warning: users unable to articulate why the schedule differs from a manual one. Testing it costs a week. Finding out in Phase 5 costs the most expensive rebuild in the project.

4.2 What to Build

Not a prototype. A **manual simulation**, because automating it would beg the question.

Produce two day plans for a realistic scenario — a professional with 8 meeting-heavy hours, three active goals, and eight tasks of mixed difficulty:

Plan	Construction
Plan A — clock-ordered	Tasks placed in priority order into available time, ignoring energy
Plan B — energy-aware	Deep work in peak hours, administrative work in troughs

Include a third variant to test the strongest claim:

Plan	Construction
Plan C — Plan B with reasoning	Same schedule as B, but each block explains its placement

4.3 Validation Method

Test with 6 to 8 target users, recruited for knowledge work with genuine deadline pressure. Five or more below this number produces unreliable signal; more than eight is wasted effort.

Two-part instrument:

Part one — preference (weak signal). Show A, B, and C side by side, ask which they would choose. Expect C to win. This proves little: people prefer explanations regardless of whether the explanation is correct.

Part two — felt difference (strong signal). Show A and B only, no explanation, ask:

- 1.
- 2.
- 3.
- 4.

Question 3 is the decisive one. **If a user cannot articulate the difference, the thesis has failed** — regardless of stated preferences.

Question 4 tests the second risk: whether users believe the app knows them. If users think they could trivially build Plan A themselves, the product has not earned its complexity.

4.4 Decision Rules

Decide before running the sessions, so the result cannot be rationalised after the fact:

Outcome	Signal	Action
Thesis confirmed	≥ 6 of 8 correctly articulate the difference in Q3	Proceed to Phase 3 as planned
Thesis weak	3 to 5 of 8	Proceed, but treat Plan C's reasoning as mandatory rather than optional, and instrument Q3 from launch
Thesis failed	≤ 2 of 8	Stop. Return to product definition. Consider whether the differentiator should be something else entirely

Also record, regardless of the thesis verdict: which plan did users think was realistic, and did the reasoning change their trust in the plan? Both inform Phase 9.

4.5 Exit Criteria

- Sessions completed with at least 6 target users
 - Q3 responses tabulated
 - Verdict recorded against the decision rules
 - Written conclusion on whether the energy thesis survives
-

5. Phase 2 — Design System

Duration: 10 to 15 days

Requirements delivered: Q3 (accessibility foundation), Q5 (low cognitive load), Q13 (tone consistency — vocabulary), and the visual foundation for all screens

Depends on: Phase 1

5.1 Why Design System Before Architecture

The design system must exist before feature screens are built, because it determines what components architecture must be able to render. Retrofitting accessibility across 15 screens after they are built is roughly ten times the work. Retrofitting it into tokens and primitives before they are built is close to free.

It also forces a decision the PRD makes repeatedly and should be settled once: **the screen inventory stays at 15**. The PRD rejects a coach chat screen. Every new component must justify itself against that constraint, because a design system that makes new screens cheap to build removes the pressure that keeps the product small (Principle 8).

5.2 Token Architecture

Design tokens in three tiers, so that retheming does not require touching components.

Tier 1 — Primitive. Raw values with no meaning. A palette of colours, a spacing scale, type sizes, durations, radii.

Tier 2 — Semantic. Purpose-named. `surface-raised`, `text-muted`, `border-subtle`, `energy-high`, `priority-critical`. **This is where most of the product's meaning lives.**

Tier 3 — Component. Bound to specific components.

The semantic tier matters more than usual here for a non-obvious reason. The product communicates state constantly — scheduled, overdue, blocked, high energy, Top 3 — and each of those must be readable without colour (Q3). Semantic tokens let those states be defined once, correctly, in one place, rather than re-derived per screen with subtle inconsistency.

5.3 Colour and Status Semantics

This is where most of the design system's effort should go, because the product's emotional safety depends on it.

Requirement D9 and N6 forbid guilt-inducing language. The colour system must be equally free of punitive signalling.

State	Treatment	Never
Completed	Muted, recedes	Loud celebration
Overdue	Neutral grey, factual	Red, warning icon, exclamation marks

Blocked	Distinct but calm	Red
High priority	Accent emphasis	Red
Missed day	Absent from display	Red counter, X marks, strike-through

The specific rule: red is never used. Not for overdue, not for errors that indicate user failure, not for missed streaks. Red is reserved exclusively for genuine input errors where the user must fix something. This is a deliberate, product-level constraint that will be debated by designers, so it is recorded here as a decision, not a suggestion.

Overdue work appears neutral and factual — "3 tasks moved to your inbox" rather than "3 tasks overdue". This is not cosmetic softening; it is the mechanism that makes recovery (Journey 8.6) possible.

5.4 Typography and Density

Calm and compact. The target user checks this app dozens of times daily, so density determines whether it feels like a tool or an obligation.

- Default body size that supports extended daily reading, since the evening review and journal are text-heavy
- Tight line height for lists, relaxed for journal entries
- Hierarchy by weight and colour rather than size alone, supporting Q3 text scaling
- Support for text scaling up to 200% without content loss

5.5 Component Inventory

Build only what the 15 screens require. Resist additions.

Foundation: button, text input, text area, select, checkbox, radio, switch, toggle group, slider, date picker, time picker, stepper, chip, badge, icon.

Status and data: progress bar, score display, stat tile, sparkline, comparison bar, empty state, error state, loading state.

Navigation: tab bar, back header, sheet, bottom sheet, inline disclosure, step indicator.

Product-specific: day timeline, schedule block, energy curve editor, task card, goal card with progress, Top 3 selector, capture button, focus session control, review question card, journal entry block, insight card with evidence citation, replan diff.

Pattern library: one-tap acceptance, undo replacing confirmation (Interaction Rule 7), reasoning disclosure, bulk selection, gentle slippage message, coach message with dismissal, weekly review decision row.

The last two deserve emphasis. The **replan diff** must show what was cut, what was pushed, and why (requirement D7) in a single glance. The **coach message with dismissal** must always offer at least two responses including dismissal (requirement A8). Both encode product principles directly into components, which means the principles survive implementation pressure.

5.6 Accessibility Baseline

Non-negotiable, and verified rather than assumed.

Requirement	Verification
Screen reader on every screen	Manual test plus automated checks
Complete keyboard navigation	Every interactive element reachable and operable
Text scaling to 200%	No content loss, no overlap
No colour-only meaning	Every state distinguishable without colour
Contrast meets AA	Automated check across all token pairs
Focus visible on every element	Keyboard traversal test
Touch targets adequate	Verified on device, not just in design

Run these against the component library before screens are built, so every screen inherits compliance rather than requiring individual audits.

5.7 Tone and Copy System

Requirement Q13 demands consistent, non-judgemental language (PRD Section 11.1). This becomes a documented vocabulary, not an aspiration.

Instead of	Write	Reason
"3 tasks overdue"	"3 tasks in your inbox"	Overdue implies failure
"You failed to complete your Top 3"	"Your day got reshaped"	Failure framing
"Don't break your streak!"	Nothing at all	Streak pressure is banned (PRD 7.3)
"You haven't journaled in 5 days"	Nothing at all	Guilt trip
"Great job hitting your goals!"	"Goal 1 moved forward"	Specific over generic praise
"You are avoiding this project"	"This project has had no movement in 3 weeks"	Inference stated as fact is banned (PRD 13.2)

Review every user-facing string against this list before release. Copy review is a phase gate, not a final polish step.

5.8 Deliverables

Token library in three tiers, semantic status model with the red-is-banned rule encoded, component library with accessibility verified, pattern library, tone and copy system, and a documented rule that any new screen requires justification against the 15-screen inventory.

5.9 Exit Criteria

- All components meet Q3 accessibility criteria, verified not assumed
 - Text scaling to 200% verified on the day timeline, the densest screen
 - Tone and copy system documented and reviewed
 - Screen inventory confirmed at 15, with no additions
 - A complete design of the day timeline exists and is walkable, since it is the densest and most-used screen and will expose system weaknesses early
-

6. Phase 3 — Architecture

Duration: 5 to 8 days

Requirements delivered: Q4 (offline), Q7 (privacy), Q8 (transparent AI), Q9 (durability), Q10 (cross-device), Q11 (resource), Q14 (graceful degradation)

Depends on: Phase 2

6.1 Approach

The architecture is described as **decisions and consequences**, not as technology choices. Stack-agnostic architecture first; a concrete stack recommendation follows in Appendix A and can be discarded without invalidating this section.

Each decision below states the choice, the reason, the consequence accepted, and what it forecloses.

6.2 Decision 1 — Offline-First, Not Offline-Capable

Decision: The device holds a complete working copy. The app is fully functional with no connection and synchronises when one returns.

Reason: Q4 is a Must. More importantly, it reflects reality for this audience. Commuting, travelling, and focus sessions are all situations with poor connectivity, and the evening review in particular happens at the end of a day when people are often out. An app that fails on the underground is an app that loses its habit.

The PRD already narrowed this correctly: Q4 covers capture, planning, focus, and review. Full offline replication is explicitly out of scope in PRD Section 7.2.

Consequence accepted: Conflict resolution becomes a permanent concern rather than a future problem. If a user edits a task on a phone and a laptop while offline, something must give.

Forecloses: Any design where the server is authoritative. That is a real loss of simplicity, and it is the price of the requirement.

6.3 Decision 2 — The Scheduler Runs on the Device

Decision: Day planning is computed locally, not on a server.

Reason: Three forces point the same way. Latency: a plan must be available instantly on open (Q1). Privacy: the schedule reveals the most intimate shape of someone's working life. Resilience: the core loop must work with no connection, per Decision 1.

Consequence accepted: Plans cannot be pre-computed for tomorrow while the device is off. Tomorrow's plan is generated when the device next runs. This is acceptable because the morning plan is a user-initiated ritual anyway (Journey 8.3), which is when the plan would be presented.

Forecloses: Generating plans for a whole week ahead without opening the app. This is requirement S13, already marked Could.

6.4 Decision 3 — AI Is a Bounded Service, Never a Planner

Decision: AI performs parsing, phrasing, and pattern description. It does not decide what goes in the day.

Reason: This is the most important architectural decision in the document, because it resolves the tension between the auto-scheduling thesis and the AI guardrails.

The scheduling algorithm is **deterministic and inspectable**. Given the same inputs it always produces the same plan, and it can always explain why each block sits where it does (requirement S7). AI may write the explanation in natural language, but never the decision.

This matters more than it appears. Requirement P3 and S7 demand visible reasoning, and PRD Section 13.2 prohibits the coach presenting inference as fact. An AI-generated schedule cannot satisfy either — it cannot guarantee consistency, and its reasoning is post-hoc rationalisation rather than the actual logic.

The AI boundary:

AI does	AI does not
Parse voice and text capture (C2, C3)	Decide task order (P2)
Choose slot placements (S2, S3)	Determine what to cut (P5)
Explain a placement in plain language (S7, A10)	Invent a reason for a placement it chose
Detect overload (A2)	Make a change without consent (PRD 13.2)
Generate journal prompts (J3, J7)	Diagnose the user (PRD 13.2)
Describe patterns in the user's data (A4, I2, I3)	State an inference as fact (Q8)

Consequence accepted: Scheduling quality now depends on hand-written algorithm quality rather than a model's capability. This is harder to improve quickly, and it is the correct trade. A schedule a user can interrogate and trust beats a marginally better schedule that changes when they do not expect it.

Forecloses: Using AI to improve scheduling by simply asking a model to plan. This is where most competitors will go, and it is the wrong choice for this product's trust requirements.

6.5 Decision 4 — The Scheduling Algorithm Is a Constraint Solver, Not a Heuristic

Decision: The day planner models the problem as constrained assignment. Tasks have requirements; time has properties; fixed commitments remove capacity. Find an assignment maximising total value subject to all constraints.

Reason: Two properties are non-negotiable. **Determinism** — the same inputs must give the same plan, or requirement P8's promise that manual overrides persist becomes meaningless. **Explainability** — the reason a task sits at 09:30 must be derivable, not guessed. A heuristic that greedily fills slots satisfies neither reliably.

A solver also makes the reasoning requirement (S7) fall out naturally: the answer includes what each placement was competing against, which is exactly what the user needs to judge whether the plan makes sense.

Consequence accepted: More complex than a greedy algorithm, and the constraints must be modelled carefully. Getting the model right is most of the difficulty of Phase 5.

Forecloses: Rapid scheduling improvements by tweaking heuristics. Changes become constraint additions, which require thought.

6.6 Decision 5 — Explanation Is a Product Feature, Not Debug Output

Decision: The scheduler's reasoning is a structured data structure from the start — which constraint bound each task, what it competed against, why it lost or won — and the UI renders it. It is not generated later by asking a model to explain a placement.

Reason: Requirements S7, P3, I3, and A10 all demand reasoning. Four separate requirements mean it is core, not a feature. Structured reasons also mean the same evidence serves the scheduler, the replan diff, and the weekly review. And it satisfies Q8 by construction, since the reasoning is a fact about the algorithm rather than a model's impression.

Consequence accepted: The data model carries reasoning metadata from the start. Cheap now, expensive later.

6.7 Decision 6 — Journal Entries Live in the Database, Not Encrypted Yet

Status: REVISED. The original decision was *local-first and encrypted at rest on the device*. That decision was written against a native app with a local SQLite store. The stack has since changed to a browser app with PostgreSQL on Neon, so the original decision could not be honoured without first building a key-management system — which is a larger project than the journal itself. Revised 3 October 2026.

Decision: Journal entries are stored as ordinary rows in the application database. Content is not encrypted client-side in the prototype.

Reason: Requirement Q7 — journal content stays private to the user — is still satisfied in the sense that matters here: no other user can read another person's entries, and nothing is ever surfaced to anyone else or used for any purpose. But it is no longer true that the provider cannot read it, and the document should say so rather than imply a protection that does not exist.

Why not encrypt now, honestly: encryption needs a key, and the prototype has no password to derive one from — authentication is a passwordless link, so there is no secret on the device. Generating and storing a key instead creates a worse failure than no encryption: **a lost key makes entries permanently unrecoverable.** For a journal, that is worse than readable text on a database you control access to. Half-built encryption is a worse outcome than none.

The trigger, so this is not forgotten: client-side encryption ships **before any real user, as opposed to a tester, writes a journal entry.** Until then, the prototype's testers are told plainly where their entries are stored.

Consequence accepted: full-text search over journal history becomes straightforward rather than a client-side concern. Requirement J5 is easier, not harder.

What this gives up: the stronger privacy guarantee in PRD Q7 and Section 14 principle 2 is deferred, not met. If the prototype graduates, this decision must be revisited before launch, and it is one of the first things a security review would examine.

6.8 Decision 7 — AI Processing Is Explicit and Disclosed

Decision: Any content sent to an AI service crosses a boundary that is disclosed at the point of use. Journal content and task text are not sent for background model improvement. AI features not required for the core loop can be disabled without degrading it.

Reason: Requirements Q7a, Q8, and PRD Section 13.3. Sending a user's private thoughts for training without meaningful consent would be a serious violation of the trust this product depends on. Product differentiation here is trust, so the architecture must enforce trust rather than rely on policy.

Consequence accepted: Less capable AI features than a competitor willing to send everything. Correct trade for this product.

6.9 Decision 8 — Privacy Controls Are Structural, Not a Settings Screen

Decision: Every data flow has a declared purpose and a disclosure point. The leaderboard is opt-in with data exposure limited to display name, and users who opt out are never listed, not merely unranked (requirement I10).

Reason: Requirement I11 requires opt-out to be one action and fully functional. Making that true structurally prevents future changes from breaking it.

Consequence accepted: More upfront work per feature, since each needs a privacy review.

Forecloses: Adding any social feature without revisiting the data model.

6.10 Decision 9 — Graceful Degradation Is a Per-Service Concern

Decision: Every external dependency — calendar providers, AI services, notifications — degrades independently and states plainly what is not working.

Reason: Requirement Q14. The failure mode to avoid is a calendar provider having an outage and the app silently producing a plan that ignores real meetings, which is worse than no plan and destroys trust.

Consequence accepted: Each integration needs an explicit degraded state designed, not defaulted into.

6.11 Decision 10 — Notifications Are a Bounded Resource

Decision: A global notification budget, with per-type quotas, enforced at the architectural level. Default maximum three per day (PRD Section 12.2).

Reason: Requirement N5, and the dismissal-rate metric in PRD Section 6.2.5 treats excessive notification as a defect. Architecture should make the policy structural.

Consequence accepted: Some genuinely useful notifications will be suppressed. That is the intended behaviour.

6.12 Decision 11 — Design for Small Surface, Hard to Expand

Decision: The navigation architecture encodes the 15-screen inventory. Adding a screen is a deliberate change requiring product approval.

Reason: Principle 8. Friction in adding screens is a feature — it is the enforcement mechanism for the product's core discipline.

Consequence accepted: User requests for features get deferred more often, which will occasionally frustrate users.

6.13 Decision Register

#	Decision	Forecloses	Revisit if
1	Offline-first	Server-authoritative model	Never. Core to the product
2	On-device scheduling	Pre-computed future plans	S13 promoted to Should
3	AI bounded, not a planner	AI-driven scheduling	Never. Trust requirement
4	Constraint solver	Rapid heuristic tuning	Scheduling quality proves adequate
5	Reasoning as data	Post-hoc explanations	Never. Four requirements depend on it
6	Journal in the database, not encrypted yet (revised)	Client-side encryption of entry text	Before any real user writes an entry — not a tester
7	Explicit AI disclosure	Bulk training on user data	Never. Trust requirement
8	Structural privacy	Cheap social features	Social scope revisited
9	Per-service degradation	Unified failure handling	Never. Q14 requirement
10	Notification budget	High notification volume	Never. Retention risk
11	Hard screen limit	Rapid feature expansion	Never. Principle 8

6.14 Exit Criteria

- All eleven decisions documented with consequences and revisit triggers
 - Every external service has a defined degraded state
 - AI boundary table agreed with engineering, so it cannot be eroded during build
 - Privacy review passed on the data flow model
 - Data model covers goals, projects, tasks, routines, blocks, journal entries, and structured reasoning
-

7. Phase 4 — Foundations

Duration: 15 to 20 days

Requirements delivered: G1, G3, G4, G6, G9, G10, G12, C1, C2, C3, C4, C5, C6, C9, and foundations of Q2, Q4, Q9, Q10, Q11

Depends on: Phase 3

7.1 Scope

The data layer plus capture and the goal hierarchy. No scheduling yet — this phase ends with users able to build a structure worth scheduling.

7.2 Build Order

Follow the capture path, because it is where users form their first impression.

Step 1 — Data model and local store. Goals, projects, tasks, routines, journal entries, blocks, and reasoning structures per Decision 5. Offline-first with durable writes per Q9.

Step 2 — Capture (C1 through C6). Voice and text into the inbox. This is Principle 2 in practice: one interaction from anywhere, transcription, parsing, and exactly one clarifying question when ambiguous (C5).

Capture must be interruptible. Per Q2, a user interrupted mid-capture returns to exactly where they were. This is an absolute gate, and it is tested by interrupting every capture path.

Step 3 — Inbox triage. File, merge, discard, file in bulk (C7, C8).

Step 4 — Goals, projects, tasks. G1, G3, G4, with the five-goal warning (G2), unfiled tasks (G5), progress view (G6), health flags (G7), lifecycle actions (G9), traceability (G10), goal-wide task views (G11), life obligations (G12).

Step 5 — Journal foundation. Entries attached to dates (J1), referencing the day's work (J2), with free-form writing and mood (J4). Guidance and search follow in Phase 7.

Step 6 — Cross-device sync. Per Q10, working from Decision 1.

7.3 Notes on Individual Requirements

Capture is the highest-polished flow in the product. Risk 3 identifies capture friction as the point where users are lost mid-thought, and Principle 2 makes attention the scarce resource. Do not treat this as a supporting feature. It deserves the most iteration of anything in this phase.

The five-goal warning (G2) must not block. Warn, offer to pause an existing goal, allow override. Goal sprawl is Risk 7, but a hard block would violate the forgiving-by-design principle.

Reasoning structures ship empty (Decision 5). Build the data shape now even though the scheduler arrives in Phase 5. Retrofitting reasoning metadata across stored blocks is expensive.

7.4 Exit Criteria

- Voice capture works end to end, under five seconds from one interaction
 - Interrupt safety verified across every input surface (Q2)
 - Natural text input correctly extracts action, date, time, and priority
 - Goals, projects, and tasks fully functional with working progress view
 - Journal entries save and survive app termination mid-write (Q9)
 - Sync works across two devices, including concurrent offline edits (Q10)
 - All work functional with no connection (Q4)
 - **Check-in:** live one full day of capture and structuring. Note every point of hesitation.
-

8. Phase 5 — Prioritisation and Scheduling

Duration: 15 to 20 days

Requirements delivered: P1, P2, P3, P4, P5, P8, S1, S2, S3, S4, S5, S6, S7, S9, S11

Depends on: Phase 4

8.1 Why This Is the Most Important Phase

This phase builds the core thesis. If the schedule is not believable, the product is a to-do list with a calendar attached, and users will correctly conclude they could have done this with a free app.

Every later phase — focus, review, insights, coaching — presents or evaluates the output of this phase. Defects here are inherited everywhere.

8.2 Step 1 — Prioritisation (P1 to P8)

Three frameworks (P1): Eisenhower, Impact versus Effort, and a custom model.

Ranking must be deterministic (Decision 4) and every rank must explain itself (P3). Manual overrides persist and the system does not silently revert them (P8) — this is a trust requirement and a frequent failure point in competitors.

The Top 3 (P4) selects from the ranking weighted by goal importance.

Elimination suggestions (P5) name specific tasks and state the consequence of each. Not "some work" — specific names.

8.3 Step 2 — The Constraint Model

Per Decision 4, model as constrained assignment.

Time dimension. Working hours, commute, protected personal time (S4), plus synced calendar events as immovable (S9).

Task dimensions. Duration, difficulty, deadline, priority, goal, project.

Energy dimension. Per the Phase 0 decision: declared curve, refined by observation, disclosed when adjusted, always overridable.

Break dimension. Focus sessions and recovery breaks included by design (S5), positioned to sustain energy rather than at fixed intervals.

Buffer. Unallocated capacity retained for overruns (S11), visible and labelled.

The buffer deserves emphasis. A day filled to capacity teaches the user the app is fiction the first time something overruns. Visible buffer is honest and it absorbs the reality that overruns happen.

8.4 Step 3 — Reasoning Output

Per Decisions 5 and 6, produce structured reasoning for every placement: which constraint bound the task, what it competed against, what displaced it or what displaced it.

This is a substantial part of the work and the easiest to under-prioritise, because it is invisible until it is missing. Requirements S7, P3, I3, and A10 all depend on it. Budget it explicitly.

8.5 Step 4 — Editing and Regeneration

Drag, resize, move, reject (S6). The system absorbs changes and re-plans the remainder, showing what moved and why.

Regeneration (S10) must preserve completed work and offer to reapply manual changes.

8.6 Step 5 — Calendar Integration

Two-way sync with all three providers (S8). Degradation per Decision 9: when a provider is unavailable, state what is not syncing, and never silently produce a plan that ignores real meetings.

8.7 Test Scenarios

Scheduling is the most algorithmically demanding part of the product. Test against realistic density, not toy cases.

Scenario	Tests
8 meeting-heavy hours, 8 mixed tasks	Realistic load, the common case

Deadline on three tasks the same day	Contention, S3, elimination (P5)
Task longer than any available gap	S4 boundaries, honest failure
Every day fully booked	Buffer behaviour, honest failure
Overloaded day, 14 hours of work in 8	Overload detection (A2), what gets cut
User disagrees with every placement	Override (P8), whether they can fight it
Calendar provider down	Degradation (Decision 9), no silent wrongness
Shift worker with irregular hours	PRD Open Question 7

The last two are the ones that get skipped and should not be.

8.8 Check-In — Extended

This phase gets an extended check-in, because the stakes are highest.

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Decision rule: if more than half the blocks get moved, the scheduler needs another iteration before proceeding. Proceeding with a distrusted scheduler means every later phase is built on an unstable foundation. Do not rationalise past this.

8.9 Exit Criteria

- Three frameworks implemented, each deterministic, each explaining its ranking
- Day plan generated respecting all constraints, with verified buffer
- Energy matching working and disclosed
- Reasoning shown for every block and traceable to real algorithm output
- Manual overrides persist across regeneration
- All three calendar providers connect and sync
- Fewer than 30% of blocks moved across three real workdays
- Replan diff shows what was cut, what was pushed, and why (D7 groundwork)
- No scenario silently produces a wrong plan

9. Phase 6 — The Daily Loop

Duration: 15 to 20 days

Requirements delivered: D1, D2, D3, D4, D5, D6, D7, D8, D9, D11, F1, F2, F3, F5, F6, N1, N2

Depends on: Phase 5

9.1 Why This Phase Defines the Product

The PRD's own failure analysis is that users abandon at the first difficult week. The daily loop is where that battle is fought or lost. Journey 8.6 (recovery from a missed day) is the highest-risk moment in the entire lifecycle, and its requirements sit here.

9.2 Step 1 — Morning Plan (D1, D2, D3)

Prompt at the user's chosen time. Present the schedule, the Top 3 with reasoning, and a one-tap energy log.

Hard limit: five minutes (D3). Measure it, do not estimate it.

The energy log is one tap and it does real work. Per the Phase 0 decision it feeds the day's scheduling, so a habit is being built rather than a form being filled in.

9.3 Step 2 — Focus Mode (F1, F2, F3, F5, F6)

One action from a block (F1). Notifications silenced (F2), distracting apps blocked (F3).

F3 is the one to check early. Blocking arbitrary applications is platform-limited. PRD Open Question 6 asks whether native capability is required. Investigate this in the first days of the phase. If full native blocking is impossible on a target platform, the honest options are a guided setup flow, an allowlist of known distracting apps, or platform-specific limitations — and PRD Section 11.3 already commits to explaining what is blocked before it happens.

On completion, log what was accomplished and whether the estimate was accurate (F5), and feed future estimates (F6).

9.4 Step 3 — Evening Review (D4, D5, D6, D11)

Three to five questions (D5). Completion status pre-filled from actual day data — the user should confirm, not transcribe.

Questions must reference the actual day: a task that slipped, a meeting that ran long. Generic prompts produce generic answers, and Risk 8 identifies week-two abandonment.

Hard limit: five minutes (D6). Every question answerable in one tap or one line.

The review must feed tomorrow's scheduling (D11). This is what makes it reflection rather than data entry.

9.5 Step 4 — Replan (D7, D8, D9)

When the day is not completed, propose a revised plan showing what was cut, what was pushed, and why (D7). One tap to accept, or adjust before accepting (D8).

D9 is a design constraint, not a feature. No punishment, no warnings, no streak loss on unrelated items. Verified by full copy review against the tone system.

Design the replan diff carefully. This screen carries the emotional weight of the product, and it is the moment Principle 4 is either honoured or broken.

9.6 Step 5 — Notifications (N1, N2)

Block-start nudge (N1), suppressed if already focused or started manually.

Day-falling-behind alert (N2), at most once per day, only on material divergence, worded neutrally and offering a replan rather than reporting failure.

Both within the three-per-day default budget (Decision 10).

9.7 Step 6 — Recovery Experience

Not a separate feature. The behaviour of the app when the user returns after absence.

Requirements: no streak warnings, no backlog wall, no missed-day counter. Welcome without reference to the gap. Present today only. Backlog summarised neutrally, never displayed as a wall.

PRD Section 11.3 requires first return after long absence to show today only. Implement this deliberately — it is the concrete mechanism for Success Criterion 10 and the mitigation for Risk 1.

9.8 Step 3 Check-In — Extended

- 1.
- 2.
- 3.
- 4.
- 5.

Step 4 is the most important test in this phase and the easiest to skip. You cannot evaluate your own recovery experience, because you know it will be gentle.

9.9 Exit Criteria

- Morning plan completable in under five minutes, measured
 - Evening review completable in under five minutes, measured, with questions referencing actual day data
 - Focus mode starts in one action, silences notifications, blocks apps or states honestly what it cannot block
 - Replan works, showing what was cut, what was pushed, and why
 - Skip a day and return: no guilt signals anywhere, verified by walking every screen
 - Notifications within the daily budget
 - Another person completes a day unassisted without hesitation
 - **Check-in:** two full days lived, plus a deliberate miss-and-return
-

10. Phase 7 — Routines and Reflection

Duration: 10 to 12 days

Requirements delivered: R1, R2, R3, R4, R5, R6, J3, J4, J5, J6, J7

Depends on: Phase 6

10.1 Why Now

Routines belong inside the planner, not beside it. PRD Section 7.3 explicitly rejects a separate habit tracker, so routines require a working scheduler to schedule against. The journal earns its place once the daily loop produces the events worth reflecting on.

10.2 Routines (R1 to R6)

All four cadence options (R2). Scheduled into the day plan like any task, respecting the same constraints (R3).

Consistency tracked per routine (R4). **Slippage surfaced gently** (R5) — "three skipped this month, adjust or pause?" Never failure framing. Routines link to goals (R7) and streaks are offered with consent, never imposed (R8). Note the PRD discrepancy: the v1 draft says streaks exist for daily rituals, while v2 limits them to routines, consistent with Section 7.3 rejecting streak pressure on rituals. **Treat v2 as authoritative.**

Pause without losing history (R6).

10.3 Journal (J3 to J7)

Guided prompts referencing the actual day (J3). Free-form writing always available with no prompt required (J4) — prompting must never feel compulsory.

Search by date, mood, goal, and text (J5). **This was specified as client-side work under the original Decision 6, which required an encrypted journal. That decision has been revised — see Section 6.7 — so search is ordinary database work instead.** Worth revisiting if client-side encryption is reinstated, because the answer changes back.

Week and month grouping (J6).

Coach follow-up questions (J7): asked when an answer is superficial, withheld when complete. Interrogating a complete answer is a failure mode worth testing for explicitly.

10.4 Check-In

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

11. Phase 8 — Progress and Insights

Duration: 12 to 15 days

Requirements delivered: I1, I2, I3, I4, I5, I6, I7, I9, I11, I13, G7, G8

Depends on: Phase 6

11.1 Why This Phase Is Where Value Is Proven

Everything before this is infrastructure. This phase is where users discover whether the effort is working. Requirement I4 leads with course-correction decisions, not statistics, because the point is to change what happens next rather than to display numbers.

11.2 The Score (I6)

Per the Phase 0 decision, the formula is defined there and must be implemented exactly as specified, because I6 requires it to be reproducible from underlying task data and I3 requires insights to cite their evidence.

The formula in one sentence, shown to the user:

Your consistency is the share of your scheduled work you completed that moved one of your active goals forward.

Ritual adherence is tracked privately for retention analysis but never appears in a user-facing score.

11.3 Strengths and Weaknesses (I7)

Equal prominence. **Stated factually without hedging** (PRD Section 11.1). The coach should report uncomfortable truths plainly.

This requires a data-visibility decision: naming "you complete 40% of deep work but 90% of admin" is far more useful than a summary figure, and is more likely to be uncomfortable. Ship the specific version. PRD Section 13.4 permits plain reporting and prohibits softening.

11.4 Insight Feed (I2, I3)

Automatic, no user effort. Every insight cites the behaviour behind it, and the minimum data threshold is disclosed — "based on 12 days of data". Where data is insufficient, say so and state what is needed. Fabricating a pattern to fill space is banned by PRD Section 13.2 and directly undermines the feature.

11.5 Weekly Review (I4, I5)

What advanced, what stalled — named specifically, not counted — the consistency score, insights, and course correction.

Each goal carries an explicit action: continue, adjust, or drop. Dropping requires confirmation and states what history is preserved.

The review must lead with decisions, not statistics. A weekly review that opens with charts and ends with a list gets skimmed and skipped (Risk 12).

11.6 Goal Health (G7, G8)

Stalled goals surfaced neutrally with reschedule, rewrite, or drop. Contention warnings when goals compete for time before deadlines.

11.7 Streaks (I9)

Routines only. **A missed day pauses rather than destroys.** Copy contains nothing implying loss. PRD Section 7.3 rejects streak pressure on the daily rituals entirely.

11.8 History (I13)

Any past week's review, score, and insight set retrievable.

11.9 Check-In

- 1.
 - 2.
 - 3.
 - 4.
 - 5.
 - 6.
-

12. Phase 9 — Coaching Layer

Duration: 10 to 12 days

Requirements delivered: A1, A2, A3, A4, A5, A8, A10, and P5 groundwork

Depends on: Phase 8

12.1 Why Last, and Why It Is the First Thing to Cut

The coach is valuable but not load-bearing. Everything it does is an enhancement to work the product already does. Per Section 2, if time runs short this phase is thinned before anything else is touched.

The good news is that this phase is cheaper than it looks, because the infrastructure is already built. Decision 5's structured reasoning means the coach explains real algorithm output rather than inventing explanations. Requirement A10 is satisfied by construction.

12.2 What the Coach Does

Morning briefing (A1). Under 150 words, spoken or written. Summarises the day, the Top 3, and anything needing a decision.

Overload detection (A2). Before the user commits to the day. Names specific tasks, never categories. This is deterministic logic operating on the schedule, not AI inference.

Journal interrogation (A3). Probing follow-ups when answers are superficial. Withheld when complete.

Pattern recognition (A4). Across weeks. Minimum data threshold before stating anything, and the threshold is disclosed.

Course correction (A5). In the weekly review. Cites inactivity. Leaves the decision entirely with the user.

Silence by default (A7). On a well-functioning day with nothing notable, no commentary at all. **Test this explicitly** — a coach that manufactures observations to appear useful is worse than no coach, and it is the most likely way this phase fails.

Always dismissible (A8). Every message offers at least two responses including dismissal. Dismissed suggestions do not reappear.

12.3 Prohibited Behaviour

PRD Section 13.2 and 13.4 in full. No diagnosis. No manipulation. No fabricated data. No unrequested generation. No inference stated as fact. No decisions on the user's behalf.

One worth restating: **the coach may suggest resting, but never suggests increasing activity to improve a rank** (PRD 13.2). If the leaderboard ships later, this must be enforced.

12.4 Check-In

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

13. Phase 10 — Onboarding and Launch Readiness

Duration: 12 to 15 days

Requirements delivered: B1, B2, B3, B4, B5, B6, B7, N3, N4, N5, N6, A7, A9, J8, and the whole of Q1 through Q14

Depends on: Phases 7, 8, 9

13.1 Onboarding (B1 to B6)

Five steps: value framing before any input, then goals, schedule and energy, framework, calendar, first plan.

Under ten minutes (B6), measured. Skippable with partial data retained (B7).

The first week is deliberately conservative. Demonstrating achievable completion is more valuable than demonstrating ambition — Risk 14 identifies over-committed onboarding as a cause of first-day abandonment.

Calendar is optional with an honest explanation of what is accessed and why (B4). Skipping does not block onboarding.

Value framing comes first, never an empty form. A user should see what the product does before being asked to supply anything.

13.2 Non-Functional Requirements — Final Verification

Every Q1 to Q14 verified rather than assumed.

ID	Requirement	Verification
Q1	Responsiveness	Measured on the oldest supported device. Any operation over one second communicates meaningful progress
Q2	Interruption safety	Interrupt every input flow and return. Nothing lost
Q3	Accessibility	Full WCAG 2.1 AA audit by a third party, not self-assessed
Q4	Offline availability	Full day completed in aeroplane mode
Q5	Low cognitive load	Five first-time users complete a day unassisted
Q6	Data ownership	Complete export, readable, ungated, deletion unobstructed
Q7	Privacy by default	Third-party privacy review
Q8	Transparent AI	AI attribution, correction, and disablement all work
Q9	Durability	Kill the app mid-write, repeatedly. Nothing lost
Q10	Cross-device consistency	Two devices, concurrent offline edits
Q11	Resource proportionality	Battery measured across a full day
Q12	Tone consistency	Full copy review against the tone system
Q13	Learnability	First day completed with no documentation
Q14	Graceful degradation	Every service failed deliberately

Q3 and Q6 deserve external verification. Self-assessment of accessibility is unreliable, and data ownership is a promise the product cannot afford to break quietly.

13.3 Notification Policy (N3, N4, N5, N6)

Ritual prompts at chosen times (N3). Every category adjustable, snoozable, silentable (N4). Default maximum three per day (N5). Full copy review for guilt-inducing language (N6).

Coach mutable except essential alerts (A9).

13.4 Export (J8, Q6)

Complete export of goals, projects, tasks, routines, journal entries, and insights. Readable, human-formatted, dated, ungated.

Test by actually reading the export. If you cannot make sense of your own export file, it fails Q6 regardless of its technical validity.

13.5 Release Gate Verification

All 16 criteria from PRD Section 16 verified as verifiable, even where the target is not yet measurable pre-launch. Several are inherently post-launch metrics — Day 30 retention, for instance — and should be instrumented and confirmed as measurable rather than as met.

Be honest about which gates can be met now and which await data. Do not claim a retention target is achieved before any user has had 30 days.

13.6 Check-In — Final

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

14. Phase 11 — Beta and Validation

Duration: 4 weeks

Requirements delivered: Validates all release criteria

Depends on: Phase 10

14.1 Purpose

Nothing in Phases 1 to 10 substitutes for real users. This phase is where the product's assumptions meet reality.

14.2 Recruitment

30 to 50 users, knowledge workers with genuine deadline pressure. Below 30, retention numbers are too noisy to interpret. Above 50 exceeds what a solo developer can support while also building.

Recruit deliberately rather than widely. Target the secondary personas as well as the primary: the Ambitious Planner and especially the Overwhelmed Returner, who is the real test of whether forgiveness works.

14.3 Weekly Cadence

Week	Focus	Key question
1	Activation	Do users reach a populated first day? Where do they stop?
2	Daily loop	Do the rituals stick or collapse?
3	Missed-day recovery	Do users come back after lapses? This is the highest-risk moment
4	Progress	Do users reach weekly review and make real decisions?

14.4 What to Watch Every Week

Do not watch: task counts, session length, notification open rates. PRD Section 6.3 lists these as counter-metrics. Rising task creation with flat goal progression means a graveyard, not engagement.

Do watch:

Metric	Target	Signal if missed
Onboarding completion	≥ 70%	Setup is too long or confusing
First day planned	≥ 55%	First-run experience fails
Morning plan completion	≥ 65%	Ritual is too heavy or the plan is not trusted
Evening review completion	≥ 55%	Reflection is not landing
Recovery within 24h of a miss	≥ 65%	Forgiveness is not working
Tasks linked to a goal	≥ 80%	Spine is being ignored
Blocks accepted unchanged	≥ 70%	Scheduling quality is insufficient
Useful insight reported	≥ 50% WAU	Insight feed is filler
Dismissal rate	≤ 20%	Notification policy is being violated

Also conduct two qualitative rounds: a first-week diary study, and interviews with users who returned after a lapse. The second is the most valuable research in the phase, because it tests the product's central differentiator at the moment it matters most.

14.5 Decision Rules

Decide in advance:

Result	Action
Loop completion $\geq$ 60%	Product thesis holding. Proceed to wider release
Loop completion 40 to 60%	Loop works but is heavy. Investigate friction before scaling
Loop completion below 40%	The daily loop is not forming. Do not scale. Return to Phase 6
Recovery below 50%	Forgiveness is failing, which is the predicted abandonment point. Return to Phase 6
Blocks accepted below 50%	Scheduling is the problem, not the ritual. Return to Phase 5
Insight usefulness below 25%	Insight feed is filler. Simplify or remove it

The last two matter most. Scaling before fixing the scheduler multiplies the problem across more users.

14.6 Exit Criteria

- 4 weeks complete with all metrics instrumented and visible
- Decision rules evaluated and acted on
- Qualitative research complete, including lapse-and-return interviews
- Written assessment of whether the thesis survived contact with real users

15. Phase 12 — Post-Launch

Duration: Ongoing

15.1 Leaderboard Decision

Deferred per Phase 0. Revisit only when Phase 11 shows Day 30 retention above 25%, which establishes a population worth ranking.

Before building, answer: does the market support it? An empty leaderboard is worse than none. Given that consistency scoring rests on a contested definition, the honest default may be to ship this never. PRD Section 7.3 rejects real-name ranking outright; if it ships at all, it is display names and opt-in.

15.2 V2 Candidates

Deferred requirements, in the order they become most valuable:

Requirement	Value if promoted	Condition for promotion
S13 — multi-day planning	Users plan on Sundays	Users complain about replanning daily

J6 — week and month journal views	Pattern recognition over time	Journal usage high enough to justify
R7 — routines linked to goals	Goal progress includes behaviour	Users ask why routines do not count
B8 — import from other tools	Reduces switching friction	Launch cohort complains about manual entry
A9 — coach muting	Users overwhelmed by coaching	Coach dismissal above 30%
G12 — life obligations	Serves non-work-heavy users	Irregular-hours research (PRD Q7) completes
F4 — energy-based breaks	Better energy management	Scheduling proven and stable
P7 — custom frameworks	Power users	Secondary persona demand
F8 — ambient sounds	Marginal	Only if trivially cheap
J9 — journal images	Emotional expression	Journal usage high
I12 — own rank when opted out	Fairness nicety	Only if leaderboard ships

15.3 Standing Priorities

Permanent, not phase-based:

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

16. Requirement Coverage Summary

16.1 By Phase

Phase	Requirements
0 — Blockers	None. Resolves PRD Open Questions 1, 2, 3, 4, 5, 8
1 — Thesis validation	None formally. Validates S2, I1, Success Criterion 3
2 — Design system	Q3, Q5, Q13 foundation
3 — Architecture	Q4, Q7, Q8, Q9, Q10, Q11, Q14
4 — Foundations	G1, G3, G4, G6, G9, G10, G12, C1–C6, C9, Q2, Q4, Q9–Q11
5 — Prioritisation and scheduling	P1–P5, P8, S1–S7, S9, S11

6 — Daily loop	D1–D9, D11, F1–F3, F5, F6, N1, N2
7 — Routines and reflection	R1–R6, J3–J7
8 — Progress and insights	I1–I7, I9, I11, I13, G7, G8
9 — Coaching	A1–A5, A8, A10
10 — Launch readiness	B1–B6, N3–N6, A7, A9, J8, Q1–Q14
11 — Beta	Validation of all
12 — Post-launch	I10, I12, and deferred Should and Could requirements

16.2 By Priority

All 82 **Must** requirements land by end of Phase 10.

Priority	Total	Phases 0–10	Phase 12
Must	82	82	—
Should	32	24	8
Could	8	0	8

Eight **Should** requirements defer to post-launch: S10, S12, S13, G8, G11, D10, N7, and C7 or C8 where triage proves sufficient without bulk operations. Confirm during Phase 11.

All 8 **Could** requirements defer. This is intentional — none is required by the daily loop, and each adds surface area against Principle 8.

16.3 Requirement Discrepancies Between PRD Drafts

The v1 Markdown draft and v2 HTML differ in three places. **v2 is authoritative**. Recorded so the differences do not surface as bugs later.

Item	v1 (PRD.md)	v2 (FocusFlow_PRD_v2)	Resolution
Non-functional IDs	Q1–Q12 plus Q7a	Q1–Q14, no Q7a	Use v2 numbering. v1's Q7a became v2's Q8
Streaks scope	I8: "daily rituals and routines"	I8: routines only, offered with consent	v2. Consistent with v2 Section 7.3 rejecting streak pressure on rituals
Coach surface	Not specified	Explicitly no chat screen (Appendix C)	v2. Coach works in context

17. Risk Register for the Plan

Risks of executing this plan, as distinct from product risks in PRD Section 17.

#	Risk	Severity	Mitigation
1	Phase 5 scheduling takes three times the estimate	High	Phase 1 validates the thesis cheaply first. If the solver proves too slow, cut features rather than extend the phase
2	Solo developer capacity is optimistic	High	Phase gates mean you can stop after any phase with something usable. Never carry known defects forward
3	AI scope creeps during Phase 9	Medium	Decision 3's AI boundary table is agreed in Phase 3 precisely so it cannot be eroded during build
4	F3 app blocking is platform-impossible	Medium	Investigate in the first days of Phase 6, not at the end. PRD 11.3 already commits to honest capability statements
5	Design system grows to serve imagined needs	Medium	Screen inventory capped at 15. New components require justification
6	Cross-device sync becomes a project of its own	Medium	Phase 4 Step 6. If it overruns, ship single-device first and say so plainly rather than slipping silently
7	Beta recruitment is too small to be meaningful	Medium	30 to 50 minimum. Below 30, retention signal is noise
8	Check-ins become ceremonial	Medium	Step 3 of the ritual is written before any discussion. Rationalisation after the fact is the failure mode
9	Phase 0 decisions get revisited mid-build	Medium	Decision register in Phase 3 states what forces a revisit. Deviations require an explicit register update
10	Energy data never gets good enough	Medium	Declared-plus-observed makes the first week work. Accept that week four is better than week one rather than expecting accuracy sooner

18. Appendix A — Concrete Stack Recommendation

This appendix is optional. Sections 3 to 17 are stack-agnostic and remain valid if this recommendation is rejected entirely.

18.1 Platform

Recommendation: React Native for iOS and Android, with a small web companion for desktop review only.

Reasons: single codebase for both mobile platforms matters enormously at solo scale; the app is overwhelmingly mobile, since capture during work and evening review happen away from a desk; focus mode needs notification and app-blocking control that only native gives; web is needed only for occasional review, so it should not get first-class design investment.

Alternative: Flutter. Comparable outcomes, different tooling. If Dart is more comfortable, this is a reasonable swap.

18.2 Storage

Recommendation: SQLite locally. **Note:** this recommendation predates the revised Decision 6 and assumed an encrypted journal table. Under the current prototype stack the database is PostgreSQL on Neon rather than local SQLite, and the journal is not encrypted client-side. This section describes the original native-app architecture and has not been re-derived for the current stack.

Reasons: relational integrity across goals, projects, and tasks; query performance over the large joins the insight feed needs; offline-first is the local database's home ground.

Alternative: a local-first sync framework. Faster to build, less control over conflict resolution, which Decision 1 made a permanent concern.

18.3 Sync

Recommendation: Operation-log sync with last-write-wins per field, plus explicit conflict detection on structured blocks.

Reasons: field-level last-write-wins handles the common case of editing different attributes on two devices; detecting and surfacing conflicts on schedule blocks respects requirement P8's promise that manual overrides persist. *(The original reason here cited "the encrypted journal syncing only as the user directs" and referenced Decision 7. It should have said Decision 6, and under the revised Decision 6 the journal is an ordinary row that syncs like any other.)*

18.4 Scheduling

Recommendation: A dedicated constraint solver module, isolated behind a single interface so it can be replaced without touching the UI.

Reasons: Decision 4's determinism and explainability requirements are testable only through a stable interface; isolating it means Phase 5 rework is contained; the reasoning output structure becomes a formal contract rather than an implementation detail.

18.5 AI Access

Recommendation: Provider-agnostic abstraction with one provider behind it.

Reasons: Decision 7 requires explicit disclosure at each point of use, which means the abstraction also knows when AI is being invoked; provider switching is a commercial decision, not an architectural one; the abstraction must never expose a decision-making capability per Decision 3.

18.6 Structure

Recommendation: Single repository, modular by feature rather than by layer.

Reason: at solo scale, layer-based structure produces navigation overhead without benefit. One module per capability area, matching the PRD's twelve areas, keeps requirements and code locatable together.

18.7 Testing

Type	Focus	Why
Unit	Scheduling algorithm, score formula, energy matching	Determinism is the property being tested
Property-based	Scheduling invariants	No hard break should ever produce an overlapping schedule
Integration	Sync conflicts, capture, calendar	Where the complexity lives
Manual	Accessibility, tone, one-day usability	Cannot be automated
Regression	Every skipped test from a check-in	Closes the loop the ritual creates

The check-in friction log should produce regression tests directly. Every defect found in a check-in becomes a permanent test.

19. Related Documents

- `FocusFlow_PRD_v2.pdf` — Product Requirements Document, the source for this plan
 - `FEATURES.md` — Feature inventory from user interviews
 - Phase 3 produces the technical design specification, out of scope here
 - Phase 2 produces the design system documentation
 - Phase 11 produces the beta findings report
-

20. Sign-Off

Role	Name	Date	Status
Product Owner			Pending
Design Lead			Pending
Engineering Lead			Pending

Immediate next action: complete Phase 0. Two decisions — the consistency score formula and the energy signal approach — block everything downstream, and both can be made in an afternoon.